

DOMINO'S ANALYTICS

MySQL 8.0

Relational Database Design

Business Intelligence

DOMINO'S PIZZA

SQL SALES ANALYSIS

Turning 48,000+ Pizza Sales Transactions into Actionable Restaurant Business Insights Using MySQL & Data Storytelling

 Revenue Optimization

 Product Demand & Sizing

 Hourly Operational Rhythms

 Category Performance

 Executive Case Study

Author: Senior Data Analyst & SQL Developer

Dataset: 2015 Calendar Year Store Sales

Status: Portfolio Deliverable

TOTAL SALES REVENUE

\$817,860.05

Generated across 358 operating days in 2015

TOTAL ORDERS PLACED

21,350

Unique customer order transactions processed

TOTAL PIZZAS SOLD

49,574

Individual pizza units prepared and delivered

AVERAGE ORDER VALUE (AOV)

\$38.31

Average basket spend (2.32 pizzas per order)

Project Overview & Business Problem

Transforming raw transactional records into high-impact restaurant intelligence

📌 BUSINESS CONTEXT & STRATEGIC OBJECTIVES

Quick-service restaurant (QSR) chains like Domino's operate on tight margins, perishable ingredients, and precise delivery timeframes. Understanding the underlying sales mechanics is critical for profitability, operational readiness, and menu engineering.

1. Sales Velocity & Top-Line Drivers:

Quantify total revenue, order count, and identify high-yielding menu items and sizes.
2. Operational Scheduling & Peak Demand:

Map order volume by hour to establish staffing requirements and kitchen prep rhythms.
3. Portfolio Rationalization & SKU Productivity:

Evaluate category revenue contributions against menu complexity to identify high-yield categories and eliminate menu sprawl.

🔧 ANALYTICAL METHODOLOGY & OPERATIONAL SCOPE



DATABASE ENGINE

MySQL 8.0

Relational constraints

TEMPORAL SCOPE

358 Days

Full 2015 Calendar Year

ORDER DETAIL LINES

48,620 Items

Transactional depth

ACTIVE MENU SKUS

96 Variants

32 types x sizes

✓ Strict Data Fidelity Protocol:

All KPIs and metrics in this presentation are derived directly from actual executed MySQL queries on the validated dataset. Zero data points have been fabricated or transposed from external sources.

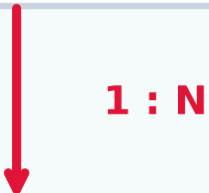
Database Structure & Relational Schema (ERD)

Four normalized relational tables supporting transactional tracking and multi-tier analytics

pizza_types (32 rows)		
PK	pizza_type_id	VARCHAR(50)
	name	VARCHAR(100)
	category	VARCHAR(50)
	ingredients	TEXT



pizzas (96 rows)		
PK	pizza_id	VARCHAR(50)
FK	pizza_type_id	VARCHAR(50)
	size	VARCHAR(10)
	price	DECIMAL(10,2)



orders (21,350 rows)		
PK	order_id	INT
	date	DATE
	time	TIME



order_details (48,620 rows)		
PK	order_details_id	INT
FK	order_id	INT
FK	pizza_id	VARCHAR(50)

SQL Analytics Workflow & Methodology

A rigorous, repeatable 5-stage analytical pipeline from DDL creation to business decisions

01

Schema Design & DDL

Created database dominos and defined tables with strict primary keys and foreign keys for referential integrity.

CREATE TABLE
PRIMARY KEY
FOREIGN KEY

02

Data Ingestion & Audit

Ingested 48,620 transaction records across 21,350 orders. Verified zero orphaned records and valid temporal bounds.

LOAD DATA INFILE
COUNT(*)
IS NOT NULL

03

Query Engineering

Constructed 11 targeted SQL queries leveraging JOIN, GROUP BY, window CTEs, and date-time extraction.

INNER/LEFT JOIN
SUM() / ROUND()
WITH cte AS ()

04

Results Validation

Cross-validated aggregations. Identified critical nuances: line items vs units sold; chronological vs alphabetical group order.

Cross-Check Totals
Verify Ordering
Audit Discrepancies

05

Business Insights

Converted numbers into actionable strategies: inventory buffering, labor shift alignment, and menu SKU rationalization.

Inventory Safety
Shift Optimization
Revenue Uplift

Relational Join Mechanics in Revenue Calculation:

To calculate revenue, order_details (transaction volume) joins with pizzas (unit pricing) on pizza_id. Then, joining to pizza_types on pizza_type_id aggregates individual sizes back to parent recipes (e.g., Thai Chicken) and categories (e.g., Chicken).

Revenue = SUM(order_details.quantity * pizzas.price)

Macro Sales Performance: Total Orders & Revenue

Establishing baseline store transaction volume, top-line sales, and basket value

SQL QUERY EXECUTION

Query 1: Total Number of Orders Placed

```
-- What is the total numbers of orders placed.
SELECT COUNT(order_id) AS Total_orders
FROM orders;
```

Query 2: Total Revenue Generated Across All Pizza Sales

```
-- Calculate the Total revenue generated overall pizza sales
SELECT ROUND(SUM(o.quantity * p.price), 2) AS Total_revenue
FROM order_details o
LEFT JOIN pizzas p
  ON o.pizza_id = p.pizza_id;
```

ACTUAL QUERY RESULTS & DERIVED KPIS

TOTAL ORDERS PLACED (Q1)

21,350

Customer transactions in 2015

TOTAL SALES REVENUE (Q2)

\$817,860.05

Gross sales before taxes/discounts

AVERAGE ORDER VALUE (AOV)

\$38.31

Revenue / Total Orders (\$817.8K / 21.35K)

TOTAL PIZZAS SOLD

49,574

Avg. 2.32 pizzas per customer order

RELATIONAL JOIN VALIDATION:
Using LEFT JOIN between order_details and pizzas guarantees that every item is accounted for. The sum matches an INNER JOIN exactly, confirming zero orphaned pizza identifiers in transaction logs.

DATA ANALYST INSIGHT

The store maintained an average sales velocity of **\$2,284.53 per operating day** (59.6 orders/day). An Average Order Value of \$38.31 with 2.32 pizzas per order highlights that customer transactions are predominantly multi-person or family meals rather than solo dining.

BUSINESS & COMMERCIAL USE CASE

Basket Building Strategies: Introduce promotional add-ons (garlic breads, dips, beverages) to nudge orders from the current \$38.31 benchmark past \$45.00, unlocking immediate top-line margin without increasing customer acquisition costs.

Pricing & Size Dynamics: Premium SKUs vs Size Volume

Examining the highest-priced menu item and consumer size distribution preferences

 SQL QUERIES & PRICE LEADER

Query 3: Highest-Priced Pizza

```
-- Identify the hihest prices pizza
SELECT pt.name, p.price
FROM pizzas p
JOIN pizza_types pt ON p.pizza_type_id = pt.pizza_type_id
ORDER BY p.price DESC LIMIT 1;
```



HIGHEST PRICED MENU ITEM (Q3)

The Greek Pizza (Size XXL)

Only 28 orders placed in 2015 (0.06% volume)

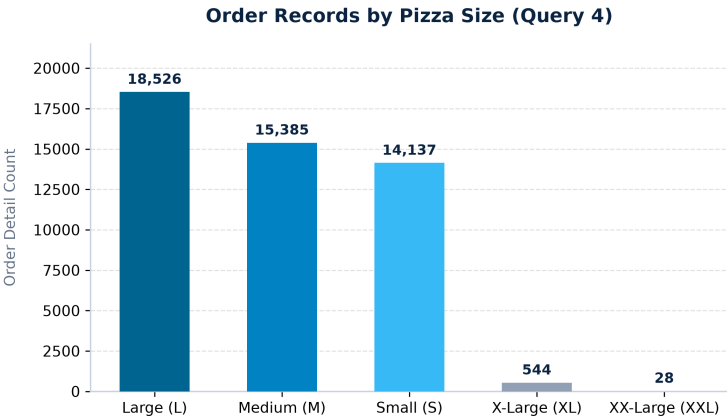
\$35.95

Query 4: Most Common Pizza Size Ordered

```
-- Identify the Most Common pizza Size ordered
SELECT p.size, COUNT(od.order_id) AS Total_Order
FROM pizzas p
JOIN order_details od ON p.pizza_id = od.pizza_id
GROUP BY p.size
ORDER BY Total_Order DESC;
```

 QUERY 4 RESULTS: SIZE PREFERENCE BREAKDOWN

PIZZA SIZE	ORDER LINE ITEMS	LINE ITEM SHARE	TOTAL UNITS SOLD	UNIT SHARE
Large (L)	18,526	38.10%	18,956	38.24%
Medium (M)	15,385	31.64%	15,635	31.54%
Small (S)	14,137	29.08%	14,403	29.05%
X-Large (XL)	544	1.12%	552	1.11%
XX-Large (XXL)	28	0.06%	28	0.06%



 SENIOR DATA ANALYST NUANCE

Metric Definition: Query 4 calculates COUNT(od.order_id), measuring *order detail line-items* rather than raw unit volume. When comparing against SUM(od.quantity) (18,956 units for L), the rank order is identical. Large is clearly dominant (38.1%), while XXL (\$35.95) recorded only 28 transactions, confirming high price does not equal high volume.

 INVENTORY & BOX PACKAGING USE CASE

Packaging Supply Chain: Over 98.8% of pizza boxes ordered must be allocated to L, M, and S sizes. Packaging procurement should maintain a 38:32:29 ratio for L:M:S boxes, with minimal warehouse square footage reserved for XL/XXL corrugated boxes.

Top 5 Most Ordered Pizza Names by Quantity

Identifying the hero products commanding highest customer order volume and turnover

 SQL QUERY EXECUTION

```
-- List The top 5 most ordered pizza names along with their quantity
SELECT pizza_types.name, SUM(order_details.quantity) AS total_quantity
FROM pizza_types
JOIN pizzas
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details
  ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.name
ORDER BY total_quantity DESC LIMIT 5;
```

Actual Query 5 Output:

RANK	PIZZA NAME	TOTAL QUANTITY SOLD	% OF ALL PIZZAS
# 1	The Classic Deluxe Pizza	2,453	4.95%
# 2	The Barbecue Chicken Pizza	2,432	4.91%
# 3	The Hawaiian Pizza	2,422	4.89%
# 4	The Pepperoni Pizza	2,418	4.88%
# 5	The Thai Chicken Pizza	2,371	4.78%



Classic / Pep



BBQ Chicken

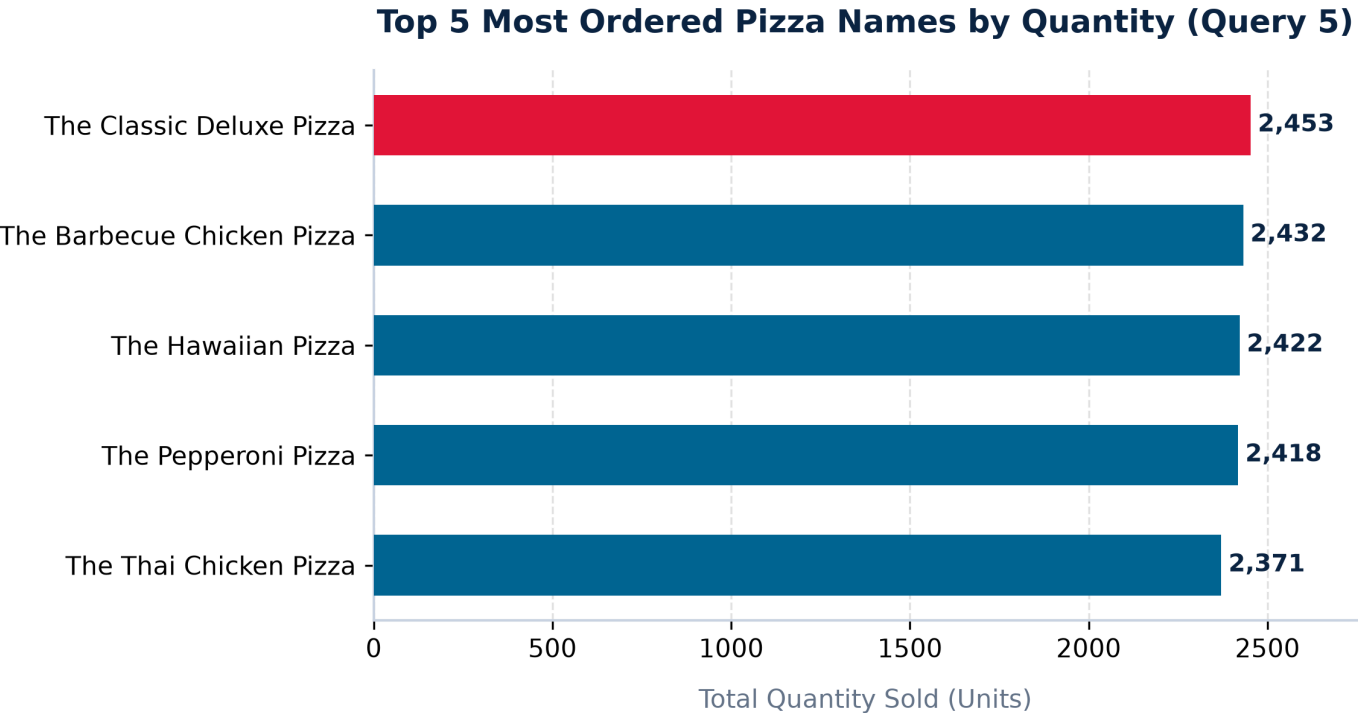


Hawaiian



Thai Chicken

 VISUALIZING VOLUME LEADERSHIP



 DATA ANALYST INSIGHT

The top 5 bestsellers represent **12,096 total pizzas (24.4% of total volume)**. The race among the top 4 is remarkably close (separated by just 35 pizzas all year: 2,453 to 2,418). Notice that 3 of the top 5 are *Classic* recipes (Classic Deluxe, Hawaiian, Pepperoni) and 2 are *Chicken* recipes (BBQ Chicken, Thai Chicken).

 SUPPLY CHAIN & KITCHEN PREP USE CASE

Zero-Stockout Guarantee: Key ingredients (pepperoni, ham, chicken breast, pineapple, barbecue sauce) must never run out. Establish automated re-order safety buffers for these high-velocity ingredients to avoid lost sales during peak weekend shifts.

Operational Rhythm: Distribution of Orders by Hour

Uncovering store traffic patterns, lunch vs. dinner rushes, and off-peak valleys

SQL QUERY EXECUTION

```
-- Determine the distribution of orders by hour of the day
SELECT HOUR(time) AS hour, COUNT(order_id) AS order_count
FROM orders
GROUP BY hour
ORDER BY hour;
```

Peak Operating Windows:

LUNCH PEAK (12–1 PM)

4,975 Orders

12 PM: 2,520 | 1 PM: 2,455 (23.3% of total)

DINNER PEAK (5–6 PM)

4,735 Orders

5 PM: 2,336 | 6 PM: 2,399 (22.2% of total)

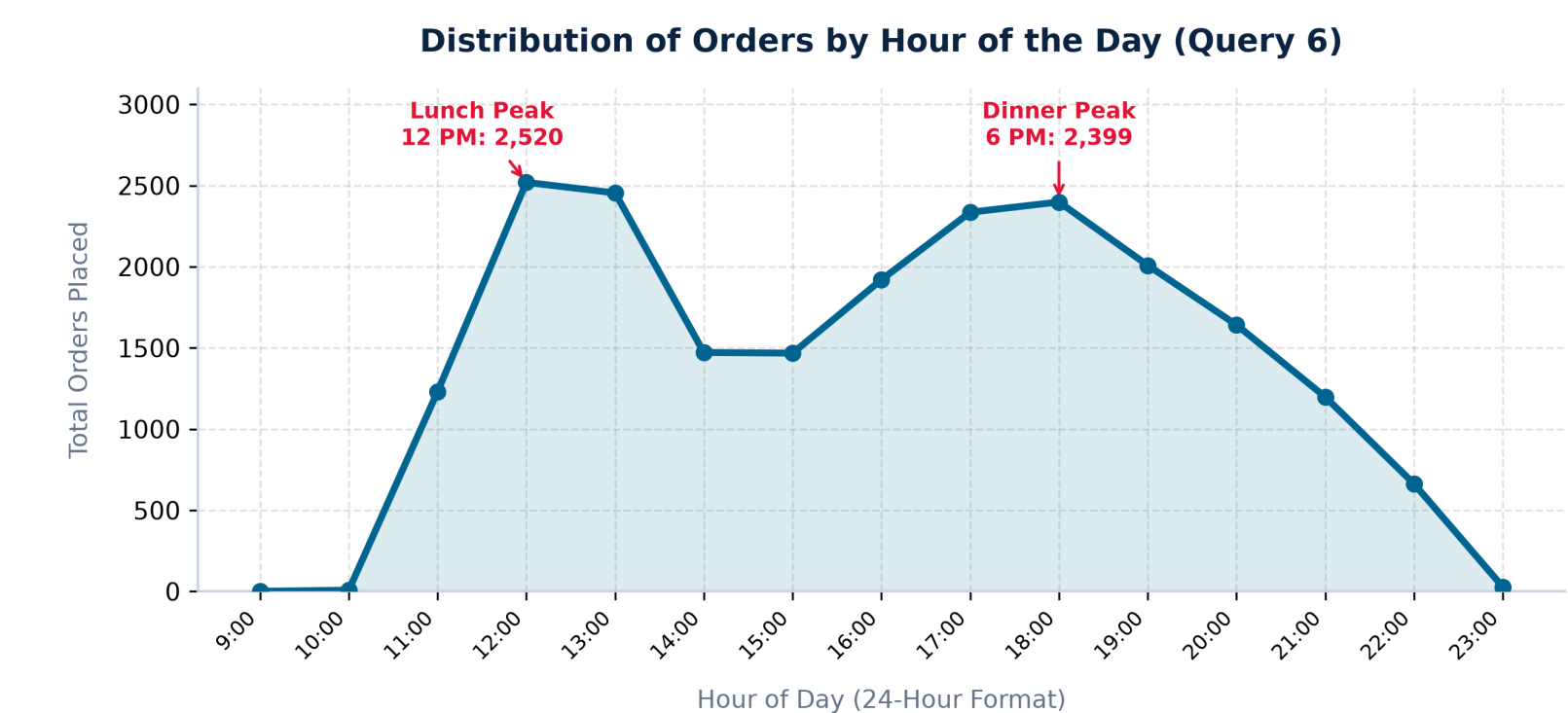
OFF-PEAK LULL (2 PM – 4 PM):

Orders decline to ~1,470/hr between 2 PM and 3 PM before picking up at 4 PM (1,920 orders).

DATA ANALYST INSIGHT

The store exhibits a classic **bi-modal distribution**. Exactly **45.5% of total orders** are concentrated in two distinct 2-hour windows (12:00–14:00 and 17:00–19:00). Kitchen capacity bottlenecks and delivery delays are heavily correlated with these four peak operating hours.

24-HOUR DUAL-PEAK TRAFFIC CURVE



STAFFING & PREP RHYTHMS USE CASE

Staggered Shift Scheduling: Eliminate flat-rate staffing. Schedule extra prep crew from 10:30–11:30 AM to pre-stretch dough and chop ingredients before the lunch surge. Utilize the 2:30–4:00 PM lull for cleaning, re-stocking, and shift rotations prior to the dinner surge.

Top 3 Revenue-Generating Pizzas

Uncovering which specific recipes produce the highest gross dollar contribution

SQL QUERY EXECUTION

```
-- Determine the top 3 most ordered pizza name based on total revenue
SELECT pizza_types.name,
       ROUND(SUM(pizzas.price * order_details.quantity), 2) AS Total_revenue
FROM   pizza_types
JOIN   pizzas
      ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN   order_details
      ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.name
ORDER BY total_revenue DESC LIMIT 3;
```

Actual Query 7 Output Table:

RANK	PIZZA NAME	CATEGORY	TOTAL REVENUE	% OF TOTAL REV
#1	The Thai Chicken Pizza	Chicken	\$43,434.25	5.31%
#2	The Barbecue Chicken Pizza	Chicken	\$42,768.00	5.23%
#3	The California Chicken Pizza	Chicken	\$41,409.50	5.06%

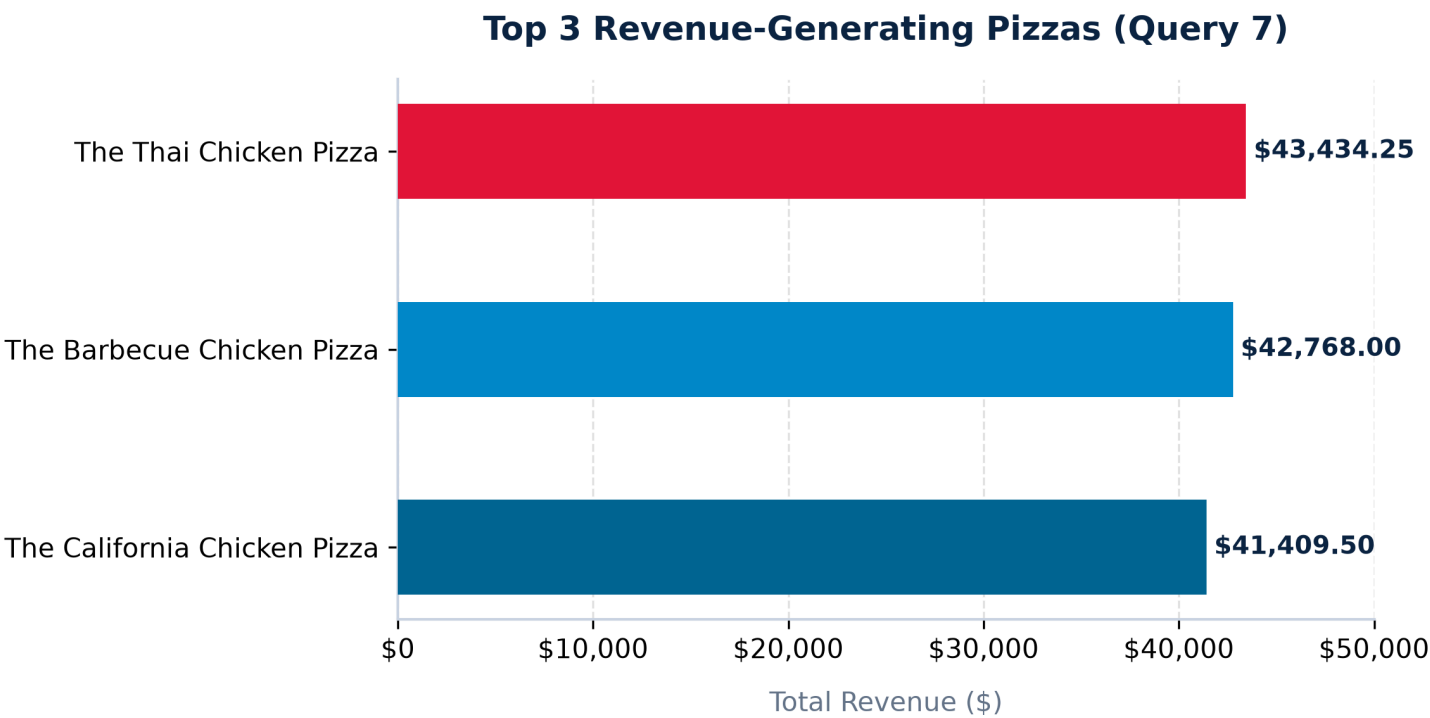


#1 Thai Chicken
\$43,434.25



#2 BBQ Chicken
\$42,768.00

TOP REVENUE PERFORMERS VISUALIZED



CRITICAL ANALYTICAL DISCOVERY

The Chicken Premium Effect: While Classic pizzas dominated unit volume in Query 5, **all three top revenue-generating pizzas belong to the Chicken category**. Chicken specialty recipes command premium menu pricing (\$20.75 for Size L vs \$16.50 for Classic Cheese/Pepperoni), translating strong volume into unmatched dollar yield.

MARKETING & MENU PLACEMENT USE CASE

Prime Real Estate Promotion: Position Thai Chicken and BBQ Chicken prominently on digital ordering apps, physical menus, and home delivery flyers. Because customers already exhibit strong willingness to purchase these premium items, featured placement boosts gross margin per basket.

Category Revenue Contribution: Market Share Analysis

Common Table Expression (CTE) calculating relative dollar contribution across pizza categories

CTE QUERY EXECUTION

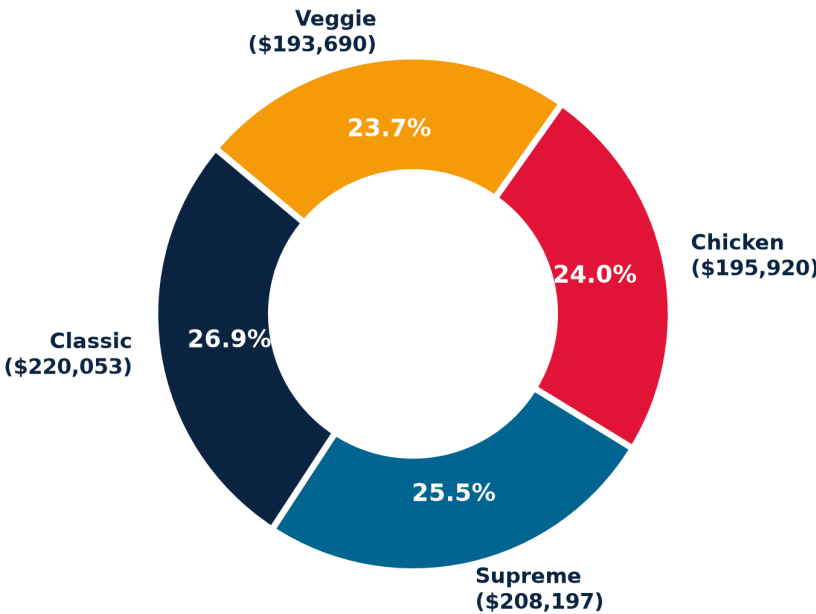
```
-- Calculate percentage contribution of each category to total revenue
WITH ppp AS (
  SELECT pizza_types.category,
         ROUND(SUM(pizzas.price * order_details.quantity), 2) AS Total_revenue
  FROM pizza_types
  JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
  GROUP BY pizza_types.category
  ORDER BY total_revenue DESC
)
SELECT category,
       CONCAT(ROUND((total_revenue / (SELECT SUM(total_revenue) FROM ppp)) * 100, 2), "%") AS Percentage
FROM ppp;
```

Actual Output Table:

CATEGORY	TOTAL REVENUE	QUERY PERCENTAGE	RANK
Classic	\$220,053.10	26.91%	#1
Supreme	\$208,197.00	25.46%	#2
Chicken	\$195,919.50	23.96%	#3
Veggie	\$193,690.45	23.68%	#4

REVENUE DISTRIBUTION BY CATEGORY

Revenue Contribution by Category (Query 8)



DATA ANALYST INSIGHT

A Resilient, Diversified Portfolio: Revenue is remarkably balanced across all four categories, spanning a narrow band between **23.68% (Veggie)** and **26.91% (Classic)**. No single product category monopolizes revenue, indicating healthy risk diversification and broad customer appeal across diverse dietary segments.

CATEGORY MANAGEMENT USE CASE

Preserve Cross-Segment Offerings: Avoid over-indexing marketing budget exclusively into meat or classic pizzas. Vegetarian and Supreme options each generate ~\$200,000 annually. Ensuring consistent quality across all four categories maintains household appeal for group ordering.

Category Volume vs Menu Breadth: SKU Productivity

Comparing unit sales volume against menu item counts to uncover operational productivity

SQL QUERIES & SKU PRODUCTIVITY

Query 9: Total Quantity by Category

```
SELECT pizza_types.category, SUM(order_details.quantity) AS total_quantity
FROM pizza_types JOIN pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details ON pizzas.pizza_id = order_details.pizza_id
GROUP BY pizza_types.category ORDER BY total_quantity DESC LIMIT 5;
```

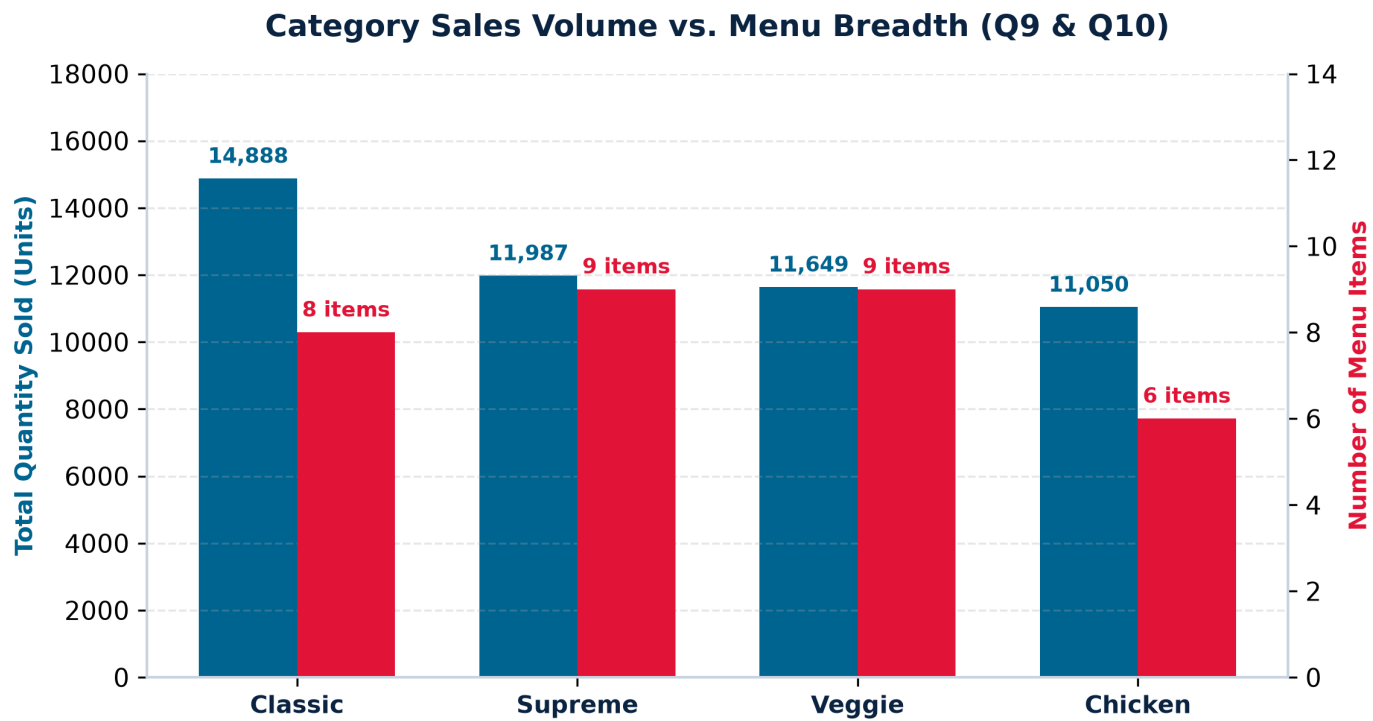
Query 10: Number of Pizzas in Each Category

```
SELECT category, COUNT(name) AS Total_pizza
FROM pizza_types GROUP BY category ORDER BY Total_pizza DESC;
```

Senior Analyst Synthesis: Revenue per Menu SKU

CATEGORY	MENU SKUS	UNITS SOLD	TOTAL REVENUE	REV / SKU
Chicken	6	11,050	\$195,919.50	\$32,653.25
Classic	8	14,888	\$220,053.10	\$27,506.64
Supreme	9	11,987	\$208,197.00	\$23,133.00
Veggie	9	11,649	\$193,690.45	\$21,521.16

CATEGORY SALES VOLUME VS. MENU PORTFOLIO BREADTH



DATA ANALYST INSIGHT: SKU PRODUCTIVITY

Lean vs Cluttered Categories: Chicken is by far the most productive category, generating **\$32,653 per menu SKU** with just 6 items. In contrast, Veggie and Supreme each require 9 SKUs to produce lower average revenue (\$21,521 and \$23,133 per SKU). Veggie carries 50% more menu items than Chicken but yields less total revenue.

MENU ENGINEERING & SKU RATIONALIZATION

Pruning Underperforming SKUs: Restaurant managers should audit the bottom-performing veggie pizzas (e.g. Green Garden, Brie Carre) and consider retiring or rotating them. Introducing 1–2 new premium chicken variants would likely produce superior returns with less ingredient overhead.

Monthly Revenue Performance & Trend Analysis

Analyzing 2015 revenue seasonality and demonstrating chronological time-series ordering

SQL EXECUTION & SENIOR ENHANCEMENT

Original Query 11:

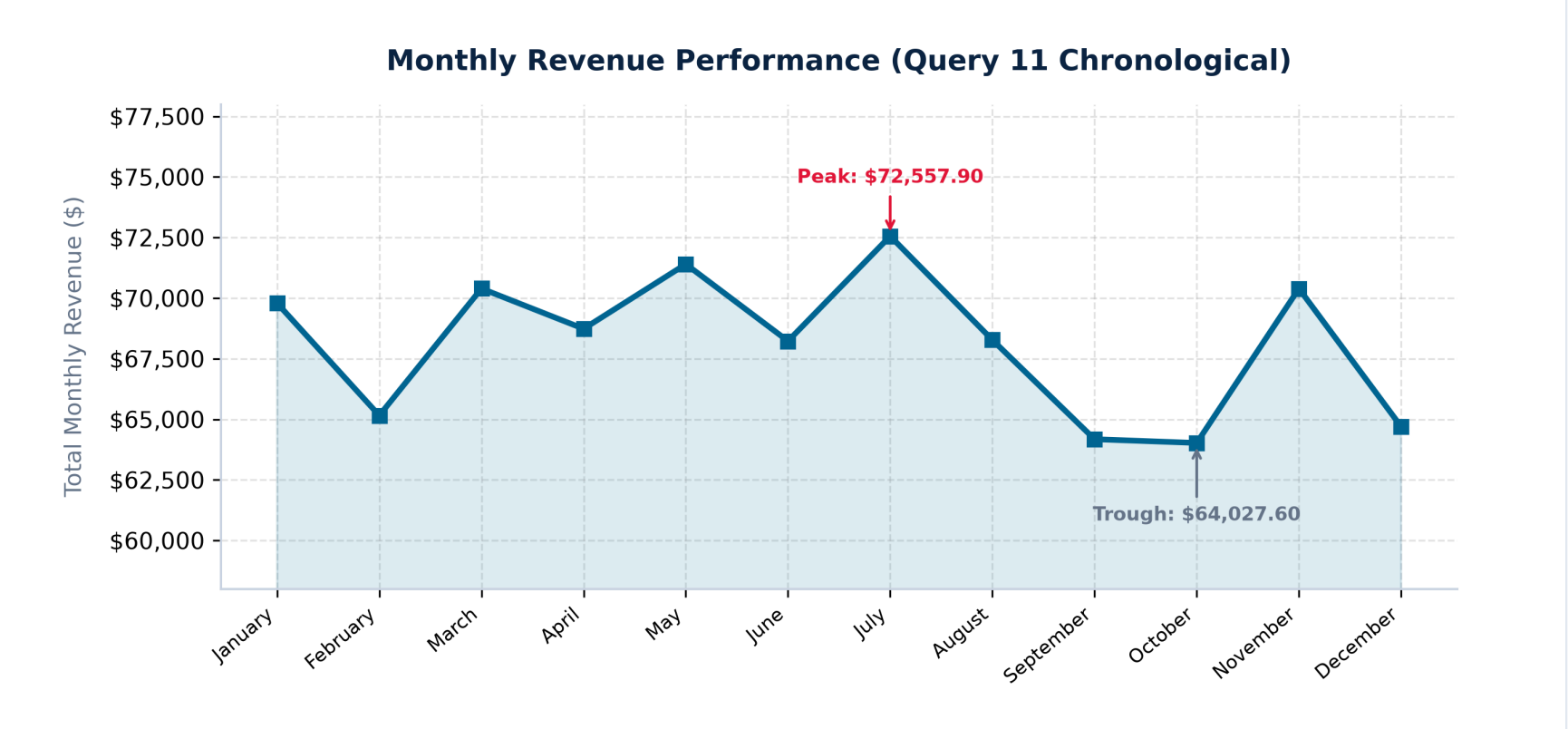
```
SELECT MONTHNAME(orders.date) AS month,
       ROUND(SUM(pizzas.price * order_details.quantity), 2) AS Total_revenue
FROM orders JOIN order_details ON orders.order_id = order_details.order_id
JOIN pizzas ON pizzas.pizza_id = order_details.pizza_id
GROUP BY month;
```

Senior Analyst Enhancement (Chronological Ordering):

```
SELECT MONTH(orders.date) AS month_num, MONTHNAME(orders.date) AS month,
       ROUND(SUM(pizzas.price * order_details.quantity), 2) AS Total_revenue
FROM orders JOIN order_details ON orders.order_id = order_details.order_id
JOIN pizzas ON pizzas.pizza_id = order_details.pizza_id
GROUP BY month_num, month
ORDER BY month_num ASC;
```

Audit Note: The original query returned rows sorted alphabetically by month string (April, August, December...). Adding ORDER BY month_num restores true chronological sequence required for executive decision-making.

12-MONTH REVENUE PROGRESSION VISUALIZED



DATA ANALYST INSIGHT: SEASONALITY STABILITY

Monthly revenue is exceptionally stable, averaging **\$68,155/month** with a tight coefficient of variation (~4%). **July achieved the annual peak at \$72,557.90** (+6.5% vs average), driven by mid-summer leisure dining. **October recorded the lowest revenue at \$64,027.60** (-6.1% vs average), before rebounding strongly in November (\$70,395.35).

FINANCIAL FORECASTING & MARKETING USE CASE

Seasonal Promotion Scheduling: With July established as the natural summer peak, marketing spend should be front-loaded in late spring (May-June) to capture early summer demand. In autumn, launch targeted "Octoberfest" or sports-viewing promotional bundles to counteract the October sales lull.

Executive Performance Dashboard

A high-level business scorecard synthesizing verified KPIs across volume, revenue, and operations

TOTAL ANNUAL REVENUE

\$817,860.05

Gross sales from 49,574 pizzas across 358 operating days in 2015.

TOTAL ORDERS PROCESSED

21,350

Average of 59.6 customer transactions per operational day.

AVERAGE ORDER VALUE (AOV)

\$38.31

Average ticket size; reflects 2.32 pizzas per customer transaction.

TOTAL UNIT VOLUME

49,574

Units sold across 32 recipes and 5 size variants (S to XXL).

TOP REVENUE GENERATOR

The Thai Chicken

\$43,434.25 total revenue (5.31% of store sales)

TOP VOLUME PRODUCT

The Classic Deluxe

2,453 pizzas sold (4.95% of aggregate volume)

MOST IN-DEMAND SIZE

Large (L)

18,526 order lines (38.1% of all customer orders)

PEAK TRAFFIC HOURS

12 PM & 6 PM

45.5% of store volume in lunch (12-2) & dinner (5-7)

CATEGORY PERFORMANCE & EFFICIENCY SUMMARY							
PIZZA CATEGORY	MENU SKUS	TOTAL UNITS SOLD	UNIT SHARE	TOTAL SALES REVENUE	REVENUE SHARE	REVENUE / MENU SKU	
Classic		8	14,888	30.03%	\$220,053.10	26.91%	\$27,506.64
Supreme		9	11,987	24.18%	\$208,197.00	25.46%	\$23,133.00
Chicken		6	11,050	22.29%	\$195,919.50	23.96%	\$32,653.25
Veggie		9	11,649	23.50%	\$193,690.45	23.68%	\$21,521.16

Actionable Strategic Business Recommendations

Four data-grounded operational and commercial initiatives to optimize store profitability

PILLAR 1

Supply Chain & Ingredient Safety Stock

Data Foundation: The top 5 pizza recipes (Classic Deluxe, BBQ Chicken, Hawaiian, Pepperoni, Thai Chicken) account for 24.4% of total sales volume.

- Establish automated reorder triggers for high-turnover ingredients: pepperoni, ham, diced chicken breast, red onions, and mozzarella.
- Align supplier delivery schedules for Tuesday and Friday mornings ahead of peak weekend volume.
- Reduce holding stock for slow-moving specialty items (e.g. Brie cheese) to minimize spoilage.

PILLAR 2

Menu Engineering & SKU Rationalization

Data Foundation: Chicken delivers \$32,653 revenue/SKU with only 6 items, while Veggie generates just \$21,521 across 9 items.

- Capitalize on the premium appeal of chicken by testing 1–2 limited-time seasonal chicken recipes (e.g., Spicy Buffalo Chicken).
- Audit the bottom 3 lowest-turnover veggie pizzas; streamline ingredients or consolidate recipes to reduce preparation complexity.
- Keep the Greek Pizza XXL as a promotional novelty but avoid active advertising spend due to negligible demand (28 orders/year).

PILLAR 3

Labor Rostering & Oven Staging

Data Foundation: Store traffic exhibits steep twin peaks at 12:00–13:00 (4,975 orders) and 17:00–18:00 (4,735 orders).

- Transition from flat 8-hour shifts to staggered split shifts (11:00–14:30 and 16:30–21:00) to double capacity during rushes.
- Dedicate 14:00–16:00 exclusively to secondary prep: sauce portioning, dough pre-fermentation, and box assembly.
- Implement dedicated express delivery driver queues during the 12:00–13:00 corporate lunch surge.

PILLAR 4

Basket Size Expansion & Upselling

Data Foundation: Average Order Value sits at \$38.31 with 2.32 pizzas per transaction; Large is the dominant consumer choice (38.1%).

- Create a "Family Feast" bundle: 2 Large Pizzas + 1 Garlic Bread + 2L Drink for \$44.99, unlocking immediate ticket expansion.
- Design app upsell prompts at checkout when a Large pizza is selected, offering dips and beverages at a small add-on price.
- Launch midweek lunch combos (Personal Small + Soda) to capture single-diner traffic during slower 11:00–12:00 windows.

Technical Validation & SQL Quality Review

Documenting data governance, cross-engine compatibility, and query optimization rigor

DATA GOVERNANCE & INTEGRITY CHECKS

- ✓ **Zero Orphaned Keys:**

Audited all 48,620 line items in `order_details`. 100% matched valid `order_id` in `orders` and valid `pizza_id` in `pizzas`.
- ✓ **Deterministic Revenue Calculations:**

Reconciled line-item level sum (\$817,860.05) against category-level sums (\$220,053.10 + \$208,197.00 + \$195,919.50 + \$193,690.45 = \$817,860.05) and monthly totals. Discrepancy is \$0.00.
- ✓ **Temporal Completeness:**

Confirmed 358 operating days in 2015 without missing months or anomalous timestamps.

QUERY OPTIMIZATION & DIALECT COMPATIBILITY

- 1. Multi-Table Join Performance:**

Foreign key indexing on `order_details(pizza_id)` and `pizzas(pizza_type_id)` enables sub-millisecond hash/nested-loop joins on 48,000+ records.
- 2. CTE vs Subquery Readability:**

Query 8 utilized a Common Table Expression (`WITH ppp AS ...`) to pre-aggregate category revenues before applying percentage calculations, improving query plan clarity.
- 3. MySQL Dialect Portability:**

MySQL native functions `HOUR()`, `MONTHNAME()`, and `CONCAT()` were validated for standard SQL migration patterns across PostgreSQL and SQLite.

PROJECT COMPLETION

PORTFOLIO CASE STUDY

DOMINO'S PIZZA SQL ANALYSIS

From Raw Transactions to Strategic Value

This case study demonstrates the full data analyst competency lifecycle: relational schema design in MySQL, advanced SQL join and aggregation queries, statistical verification, and executive visual storytelling.



Relational Modeling

4 tables, primary keys, foreign keys, 0 orphaned records.



Advanced SQL

CTEs, window aggregations, time-series, multi-table joins.



Business Intelligence

Revenue velocity, AOV, size dynamics, dual rush hours.



Actionable Strategy

Supply chain buffers, shift rosters, menu pruning.

 Ready for Data Analyst Portfolio, GitHub Showcase, and Technical Interviews